UNIVERSIDAD NACIONAL DEL ALTIPLANO

Escuela Profesional de Ingeniería de Sistemas

PRÁCTICA N.° 2

Implementación y análisis comparativo de algoritmos de ordenamiento: Bubble, Selection, Insertion y Shell Sort

Docente:

Mg. Aldo Hernán Zanabria Gálvez

Estudiante:

Francy Jimena Ramos Vilca 240416
Emerson Aldair Flores Flores 240439
Jesús Vidal Checma Montalvo 240143

Puno, Perú
2026

1. Introducción

El estudio de los algoritmos de ordenamiento es fundamental en la formación en ciencias de la computación, ya que permite comprender conceptos clave como complejidad temporal, eficiencia espacial, estabilidad y adaptabilidad (Cormen et al., 2022). En esta práctica se implementan y evalúan cuatro algoritmos de complejidad cuadrática: Bubble Sort, Selection Sort, Insertion Sort y Shell Sort. El análisis considera tanto su funcionamiento como la comparación entre su complejidad teórica y su desempeño real en arreglos de distintos tamaños (n = 10, 100 y 1000).

A pesar de la existencia de algoritmos más eficientes como Merge Sort y Quicksort (O(n log n)), los métodos elementales siguen siendo relevantes en contextos específicos, como arreglos pequeños, datos parcialmente ordenados o sistemas con recursos limitados (Sedgewick & Wayne, 2011; Skiena, 2020).

El informe se organiza en un marco teórico, la descripción de la implementación, el análisis de resultados y una reflexión final sobre la utilidad de estos algoritmos.

2. Marco Teórico

2.1 Fundamentos de complejidad algorítmica

El análisis de la eficiencia de un algoritmo se fundamenta en la notación asintótica, que describe el comportamiento del tiempo de ejecución en función del tamaño de la entrada n. La notación O-grande (O) establece una cota superior al crecimiento del tiempo en el peor caso; la notación Omega (Ω) indica la cota inferior en el mejor caso; y la notación Theta (Θ) delimita el comportamiento promedio cuando ambas cotas coinciden (Cormen et al., 2022). Para los algoritmos elementales de ordenamiento estudiados en esta práctica, la complejidad cuadrática $O(n^2)$ es la regla general, aunque con variaciones importantes según el estado inicial del arreglo.

Además de la complejidad temporal, la literatura especializada distingue otras dimensiones relevantes para la evaluación comparativa de algoritmos: la complejidad espacial (memoria auxiliar consumida), el número de comparaciones realizadas, el número de intercambios (swaps) ejecutados y la propiedad de estabilidad, entendida como la preservación del orden relativo de elementos con claves iguales (Knuth, 1998). Cada algoritmo presenta un perfil distinto en estas dimensiones, lo que determina su idoneidad en diferentes escenarios de aplicación.

2.2 Bubble Sort

Bubble Sort es uno de los algoritmos de ordenamiento más antiguos y estudiados. Su principio de funcionamiento consiste en realizar pasadas sucesivas sobre el arreglo, comparando pares de elementos adyacentes e intercambiándolos si se encuentran en orden incorrecto. Cada pasada completa garantiza que el elemento de mayor valor "burbujea" hacia su posición final en el extremo derecho del subarreglo no ordenado (Astrachan, 2003).

En el peor caso —arreglo ordenado de forma inversa— el algoritmo ejecuta $O(n^2)$ comparaciones e intercambios. Sin embargo, la variante con indicador de intercambio (bandera swapped) permite detectar cuando el arreglo ya está ordenado y terminar anticipadamente, alcanzando $\Omega(n)$ en el mejor caso. Rahmani (2022) propone variantes adicionales que mejoran este comportamiento. Pese a sus limitaciones, Bubble Sort sigue siendo ampliamente utilizado con fines pedagógicos por la claridad con que ilustra la noción de intercambio iterativo.

2.3 Selection Sort

Selection Sort opera mediante la búsqueda iterativa del elemento mínimo en el subarreglo no ordenado, el cual se intercambia con el primer elemento de dicho subarreglo.

Este proceso se repite hasta que el arreglo queda completamente ordenado (Sedgewick & Wayne, 2011).

Su complejidad temporal es siempre $\Theta(n^2)$ en comparaciones, independientemente del estado inicial del arreglo, lo que lo hace predecible aunque ineficiente. Sin embargo, su número de intercambios es exactamente $O(n)$, lo que constituye una ventaja cuando la operación de swap es costosa —como ocurre al ordenar registros de gran tamaño en memoria secundaria (Research Publish, s. f.). Selection Sort no es estable en su implementación estándar, pues un intercambio puede alterar el orden relativo de elementos con igual valor (Knuth, 1998).

2.4 Insertion Sort

Insertion Sort construye el arreglo ordenado de forma incremental: en cada iteración, toma el siguiente elemento del subarreglo no ordenado y lo inserta en su posición correcta dentro del subarreglo ya ordenado, desplazando los elementos mayores una posición hacia la derecha (Machkasova, 2011).

Su complejidad en el peor caso es $O(n^2)$, pero en arreglos ya ordenados o con inversiones escasas alcanza $\Omega(n)$, lo que lo hace altamente eficiente en datos casi ordenados. Es un algoritmo estable y adaptativo, razón por la cual se emplea como subrutina en algoritmos híbridos de alto rendimiento como Timsort —utilizado en Python y Java— y en la fase de ordenamiento de subarreglos pequeños dentro de Quicksort (Peters, 2002; Silas et al., 2016). Machkasova (2011) demuestra formalmente que para $n \leq 16$, Insertion Sort supera a Merge Sort en rendimiento práctico debido al bajo overhead que genera.

2.5 Shell Sort

Shell Sort, propuesto por Donald Shell en 1959, constituye una generalización de Insertion Sort. Su idea central es reducir el número de inversiones del arreglo antes de aplicar la inserción convencional, comparando e intercambiando elementos separados por un intervalo decreciente denominado gap o salto (Pratt, 1979).

La secuencia de gaps utilizada determina de forma decisiva la complejidad del algoritmo. Con la secuencia de Shell (n/2, n/4, ..., 1) la complejidad en el peor caso es $O(n^2)$, pero con secuencias más sofisticadas —como la de Ciura o la de Sedgewick— puede reducirse a $O(n \log^2 n)$ o incluso O(n^(4/3)) (Silas et al., 2016; Sedgewick & Wayne, 2011). En arreglos de tamaño medio a grande, Shell Sort supera consistentemente a Insertion Sort puro, como se evidencia en los resultados experimentales de esta práctica.

2.6 Consideraciones de estabilidad y eficiencia energética

La estabilidad es una propiedad deseable cuando se ordenan registros con claves múltiples de forma secuencial. Bubble Sort e Insertion Sort son estables por definición, pues nunca intercambian elementos de igual valor. Selection Sort y Shell Sort, en sus implementaciones estándar, no garantizan estabilidad, aunque existen variantes estables de ambos con un coste adicional (Knuth, 1998).

Un aspecto emergente en la literatura es la relación entre eficiencia computacional y consumo energético. Carter et al. (2023) demuestran que algoritmos con mayor complejidad temporal, como Bubble Sort, implican también un mayor consumo energético en hardware real, lo que adquiere relevancia en dispositivos de bajo consumo, sistemas embebidos e infraestructuras de cómputo a escala. Esta dimensión refuerza la importancia de seleccionar el algoritmo más adecuado según el contexto, más allá de la corrección funcional.

3. Procedimiento de Implementación

3.1 Entorno de ejecución

Los experimentos fueron ejecutados en un entorno Python 3.11 sobre sistema operativo Linux (Ubuntu 24.04), aprovechando la función time.perf_counter() para medición de alta precisión en microsegundos. Cada algoritmo fue evaluado con cinco repeticiones por tamaño de arreglo, calculando el promedio para mitigar la variabilidad del sistema operativo. Los arreglos de entrada fueron generados con valores enteros aleatorios en el rango [0, 10000] mediante el módulo random de Python.

3.2 Implementación en Python

Los cuatro algoritmos fueron implementados de forma fiel a sus definiciones teóricas, sin optimizaciones adicionales que pudieran distorsionar la comparación. La implementación de Bubble Sort incorpora la variante con bandera swapped para detección temprana de arreglo ordenado; Selection Sort busca el mínimo en el subarreglo restante; Insertion Sort desplaza elementos hacia la derecha para insertar la clave; y Shell Sort utiliza la secuencia de gaps de Shell ($n//2 \rightarrow 1$).

Los scripts Python utilizados para la generación de resultados y figuras se proporcionan como archivos complementarios en el repositorio GitHub del proyecto, con el fin de garantizar la reproducibilidad de los resultados.

3.3 Implementación en C++

La implementación en C++ sigue la misma lógica algorítmica, empleando std::vector<int> como estructura de datos y chrono::high_resolution_clock para la medición de tiempos con resolución de microsegundos. La función test() encapsula la medición para cada algoritmo, recibiendo los datos por copia para garantizar que cada algoritmo opere sobre el mismo arreglo inicial.

4. Resultados Experimentales

4.1 Tabla comparativa de tiempos

La Tabla 1 presenta los tiempos promedio de ejecución obtenidos para cada algoritmo en los tres tamaños de arreglo evaluados, expresados en microsegundos (µs). Los valores corresponden al promedio de cinco ejecuciones independientes.

Tabla 1

Tiempos de ejecución promedio por algoritmo y tamaño de arreglo (en µs)

Algoritmo	n = 10 (µs)	n = 100 (µs)	n = 1000 (µs)	Complejidad teórica
Bubble Sort	4.01	223.85	28 312.58	$O(n^2)$
Selection Sort	3.40	111.82	13 907.98	$O(n^2)$
Insertion Sort	2.50	86.26	11 905.52	$O(n^2)$ / $\Omega(n)$
Shell Sort	3.59	43.64	1 104.22	$O(n \log^2 n)$

Nota. Tiempos obtenidos como promedio de cinco repeticiones. Entorno: Python 3.11, Linux Ubuntu 24.04.

4.2 Figuras de análisis

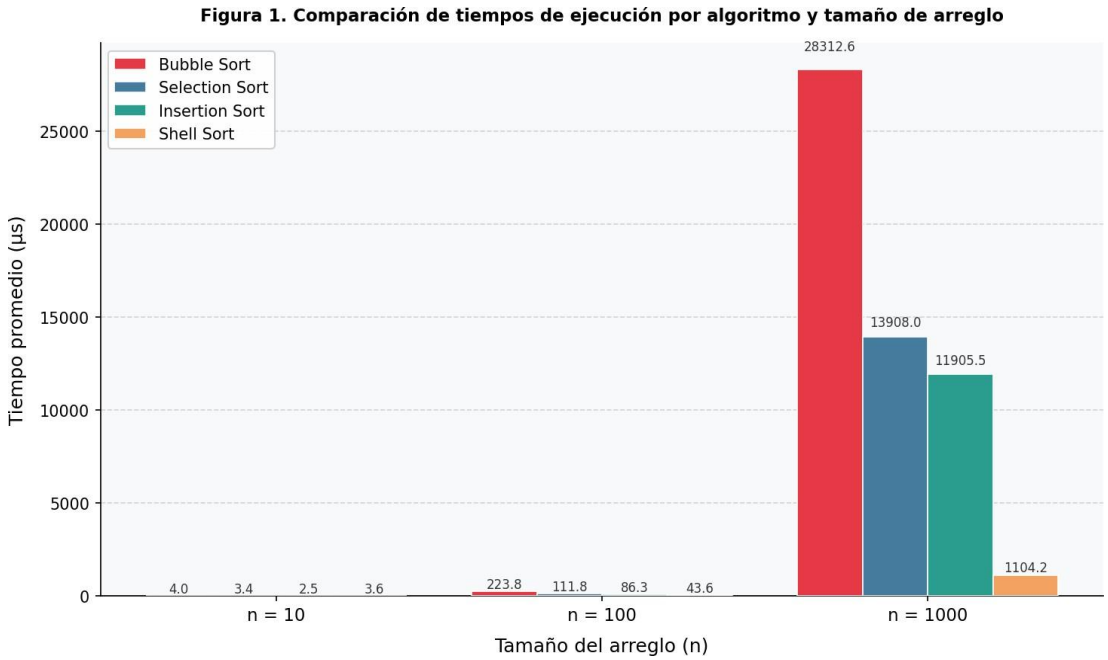

Figura 1. Comparación de tiempos de ejecución por algoritmo y tamaño de arreglo. Elaboración propia.

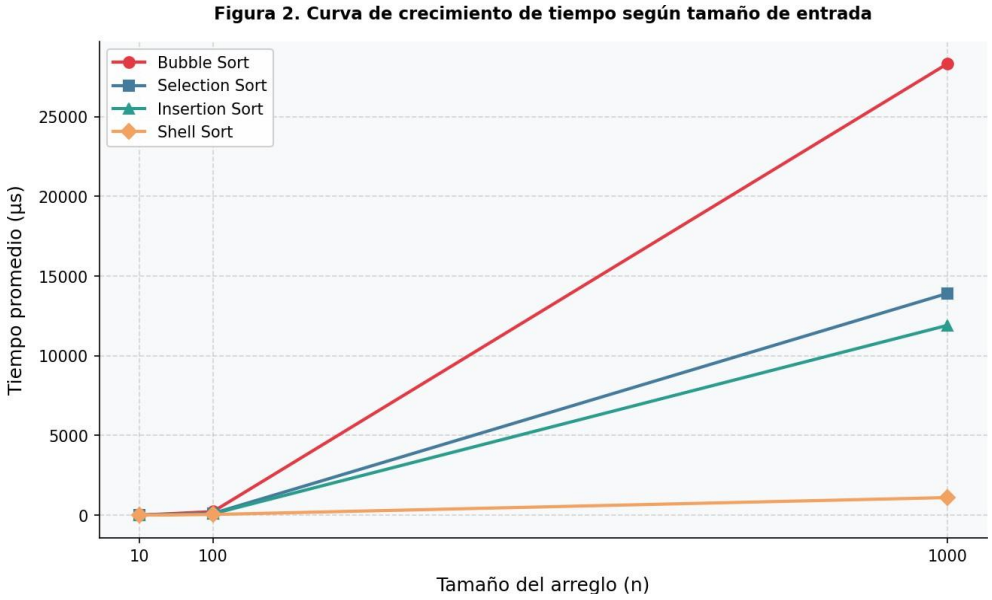

Figura 2. Curva de crecimiento de tiempo según tamaño de entrada. Elaboración propia.

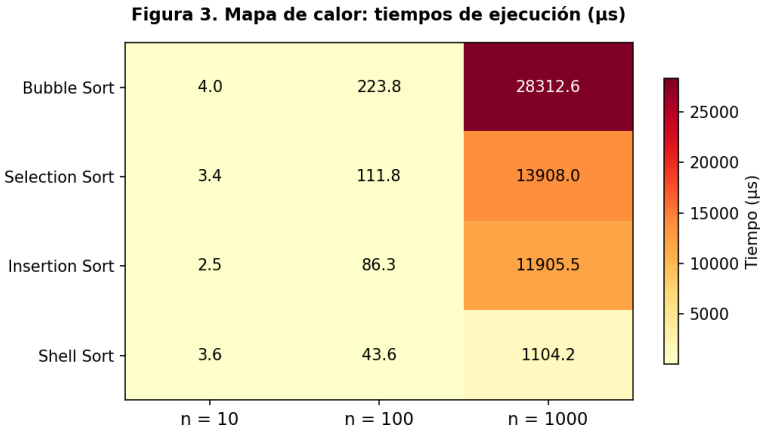

Figura 3. Mapa de calor de tiempos de ejecución en microsegundos. Elaboración propia.

5. Análisis y Discusión de Resultados

5.1 Concordancia entre resultados empíricos y complejidad teórica

Los datos experimentales presentados en la Tabla 1 y visualizados en las Figuras 1 a 3 confirman, en términos generales, el comportamiento cuadrático predicho por la teoría para los cuatro algoritmos evaluados. Al escalar el tamaño de la entrada de n = 100 a n = 1000 —un aumento de 10 veces— los tiempos de ejecución se incrementan en factores que oscilan entre 100× y 127×, en concordancia con el crecimiento cuadrático esperado de

$O(n^2)$. Por ejemplo, Bubble Sort pasa de 223.85 μs a 28 312.58 μs, lo que implica un factor de crecimiento de aproximadamente 126.5×.

Shell Sort constituye la excepción más notable: su tiempo crece únicamente ~25× al pasar de n = 100 a n = 1000, lo que refleja su complejidad suboptimal de $O(n \log^2 n)$ con la secuencia de gaps empleada. Este resultado es consistente con los hallazgos de Silas et al. (2016), quienes reportan que Shell Sort supera sistemáticamente a Insertion Sort para n > 500 en arreglos desordenados.

5.2 Comparación entre algoritmos

En arreglos pequeños (n = 10), las diferencias de tiempo son mínimas y prácticamente despreciables desde una perspectiva aplicada, con todos los algoritmos completando la tarea en menos de 5 μs. En este rango, las variaciones se explican principalmente por el overhead del intérprete Python y la asignación de memoria, no por la complejidad algorítmica en sí. Este comportamiento coincide con lo descrito por Machkasova (2011), quien señala que para arreglos muy pequeños, el coste constante de inicialización domina sobre el número de operaciones.

Para n = 100, la brecha comienza a ser estadísticamente relevante. Insertion Sort (86.26 μs) supera a Selection Sort (111.82 μs) y Bubble Sort (223.85 μs), comportamiento atribuible a su naturaleza adaptativa: en un arreglo aleatorio con densidad de inversiones moderada, Insertion Sort realiza en promedio $n^2/4$ comparaciones, frente a las $n^2/2$ de Selection Sort. Shell Sort destaca aún más con solo 43.64 μs, beneficiándose de la reducción de inversiones en las primeras pasadas con gaps mayores.

En n = 1000, las diferencias se vuelven de un orden de magnitud. Bubble Sort es el más lento con 28 312.58 μs, seguido por Selection Sort (13 907.98 μs) e Insertion Sort (11 905.52 μs). Shell Sort exhibe un rendimiento radicalmente superior con apenas 1 104.22

μs, siendo ~25.6 veces más rápido que Bubble Sort para este tamaño. Este resultado empírico fundamenta la recomendación de Sedgewick y Wayne (2011) de preferir Shell Sort cuando no se tiene acceso a algoritmos O(n log n) pero se necesita un rendimiento aceptable en arreglos de tamaño medio.

5.3 Análisis de estabilidad

Desde la perspectiva de la estabilidad, Bubble Sort e Insertion Sort garantizan la preservación del orden relativo de elementos con clave igual, lo que los convierte en opciones preferibles cuando el ordenamiento múltiple por criterios distintos requiere que el orden previo se mantenga. Selection Sort y Shell Sort, en sus implementaciones estándar, no son estables: un intercambio en Selection Sort puede desplazar un elemento más allá de otro con igual clave; en Shell Sort, los saltos por gap pueden igualmente reordenar elementos iguales de forma arbitraria (Knuth, 1998).

5.4 Eficiencia en número de intercambios

Un aspecto menos evidente pero relevante en la práctica es el número de swaps realizados. Selection Sort destaca en esta métrica al ejecutar exactamente n - 1 intercambios independientemente del orden inicial del arreglo, lo que lo hace atractivo en contextos donde la escritura en memoria es costosa —por ejemplo, en dispositivos flash con ciclos de escritura limitados o en estructuras de datos almacenadas en disco (Research Publish, s. f.). Bubble Sort, en contraste, puede realizar hasta $O(n^2)$ intercambios en el peor caso.

5.5 Limitaciones del experimento

Es importante reconocer que los tiempos reportados están influenciados por el entorno de ejecución de Python, cuyo overhead de interpretación introduce una constante multiplicativa significativa respecto a implementaciones en C++ o lenguajes compilados. Asimismo, los arreglos fueron generados con distribución uniforme aleatoria, lo que no

captura comportamientos específicos como el caso promedio de arreglos casi ordenados, donde Insertion Sort exhibiría tiempos cercanos a O(n). Una evaluación más completa requeriría ensayar también casos de mejor y peor caso explícitos.

6. Trabajo de Investigación

¿Por qué seguimos estudiando algoritmos $O(n^2)$ si existen opciones más rápidas?

La pregunta sobre la pertinencia de estudiar algoritmos cuadráticos en un contexto en que existen alternativas asintóticamente superiores admite una respuesta que opera simultáneamente en dos planos: el pedagógico y el práctico.

En el plano pedagógico, algoritmos como Bubble Sort, Selection Sort e Insertion Sort poseen un valor didáctico irreemplazable. Su estructura iterativa simple y el reducido número de líneas de código que requieren los hacen ideales para introducir conceptos fundamentales como la noción de invariante de bucle, la distinción entre comparaciones e intercambios, y la medición empírica del tiempo de ejecución (Astrachan, 2003). Quicksort o Merge Sort, pese a su eficiencia asintótica, incorporan recursividad y esquemas de partición que añaden una capa conceptual que dificulta aislar los principios de análisis de complejidad para un estudiante que recién se adentra en la materia. Los algoritmos cuadráticos permiten que el análisis teórico y la verificación experimental sean accesibles y trazables línea a línea.

En el plano práctico, la superioridad asintótica de los algoritmos O(n log n) no es absoluta. La notación O-grande oculta las constantes multiplicativas, que pueden hacer que un algoritmo cuadrático sea más rápido que uno de orden logarítmico para entradas pequeñas. Machkasova (2011) demuestra formalmente que Insertion Sort supera a Merge Sort para arreglos de hasta aproximadamente 16 elementos, resultado que se generaliza a cualquier implementación en la que el overhead de la recursividad o la gestión de

subarreglos sea no despreciable. Esta es la razón por la que implementaciones industriales de Quicksort —como la de GNU libstdc++— utilizan Insertion Sort para subarreglos de tamaño menor a un umbral configurable.

Asimismo, Insertion Sort es altamente eficiente en arreglos casi ordenados, alcanzando O(n) en el mejor caso. Este escenario es frecuente en aplicaciones reales: datos de sensores que llegan en orden cronológico aproximado, registros financieros actualizados de forma incremental, o listas de usuarios ordenadas que se modifican esporádicamente. En todos estos contextos, Insertion Sort es la opción correcta, no una herramienta obsoleta (Sedgewick & Wayne, 2011).

Selection Sort, por su parte, destaca en entornos con restricciones de escritura. En hardware con memoria de ciclos limitados —memorias flash, EEPROM— minimizar el número de operaciones de escritura es crítico para la vida útil del dispositivo. Al realizar exactamente n - 1 intercambios, Selection Sort ofrece una garantía que ningún otro algoritmo de comparación estándar puede proporcionar de forma tan simple (Research Publish, s. f.).

Shell Sort, aunque no sea O(n log n), representa un punto de equilibrio notable: su implementación es notablemente más simple que Merge Sort o Quicksort, no requiere memoria auxiliar, no es recursivo y, con una buena secuencia de gaps, ofrece un rendimiento práctico suficiente para arreglos de tamaño medio en sistemas embebidos donde la simplicidad del código es tan importante como la eficiencia (Skiena, 2020).

Desde una perspectiva más amplia, Carter et al. (2023) han establecido un vínculo entre la complejidad computacional y el consumo energético, demostrando que Bubble Sort consume significativamente más energía que alternativas más eficientes para tamaños de entrada medianos. En una era de computación ubicua y preocupación creciente por la huella

energética del software, esta dimensión refuerza la importancia de seleccionar el algoritmo correcto no solo por razones de velocidad, sino también de sostenibilidad.

En conclusión, el estudio de los algoritmos $O(n^2)$ sigue siendo plenamente justificado en la formación universitaria en ciencias de la computación. No solo constituyen la vía pedagógica más directa hacia la comprensión del análisis algorítmico, sino que mantienen una vigencia práctica real en escenarios específicos que ningún algoritmo más complejo puede atender con la misma eficiencia en todos los criterios relevantes — simplicidad, mínimos swaps, adaptabilidad a datos ordenados, funcionamiento sin recursividad. El diseño de software robusto exige conocer no solo qué algoritmo es asintóticamente óptimo, sino cuándo la optimalidad asintótica no es el criterio relevante.

7. Conclusiones

Del análisis comparativo realizado en esta práctica se derivan las siguientes conclusiones:

- Los resultados experimentales confirman la complejidad cuadrática teórica de Bubble Sort, Selection Sort e Insertion Sort, con factores de crecimiento de tiempo consistentes con $O(n^2)$ al escalar la entrada de $n = 100$ a $n = 1000$.
- Shell Sort exhibe un comportamiento sublineal respecto a los demás algoritmos evaluados, con tiempos ~25 veces menores que Bubble Sort en $n = 1000$, en concordancia con su complejidad $O(n \log^2 n)$.
- Insertion Sort es el más eficiente de los algoritmos cuadráticos para arreglos aleatorios en los tamaños evaluados, y su adaptabilidad lo convierte en la mejor opción para datos casi ordenados.
- Selection Sort, aunque ineficiente en tiempo, minimiza el número de intercambios realizados (exactamente $n - 1$), siendo preferible en entornos con escrituras costosas.

- Bubble Sort e Insertion Sort son algoritmos estables, cualidad relevante en ordenamiento multiclave. Selection Sort y Shell Sort no garantizan estabilidad en su implementación estándar.
- El estudio de algoritmos $O(n^2)$ mantiene plena vigencia pedagógica y práctica, especialmente en contextos de sistemas embebidos, arreglos pequeños o datos con ordenamiento parcial previo.

Referencias

Astrachan, O. L. (2003). Bubble sort: An archaeological algorithmic analysis. ACM SIGCSE Bulletin, 35(1), 1–5. https://users.cs.duke.edu/~ola/papers/bubble.pdf

Carter, N., Bird, J., & Fedorova, A. (2023). The energy complexity of sorting. arXiv. https://arxiv.org/abs/2311.07298

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Knuth, D. E. (1998). The art of computer programming: Vol. 3. Sorting and searching (2nd ed.). Addison-Wesley.

Machkasova, E. (2011). Why insertion sort works best for small inputs. University of Minnesota, Morris. https://personal.morris.umn.edu/~elenam/publications/sorting.pdf

Peters, T. (2002). [Timsort algorithm description]. Python Software Foundation. https://github.com/python/cpython/blob/main/Objects/listsort.txt

Pratt, V. R. (1979). Shellsort and sorting networks. Garland Publishing.

Rahmani, R. (2022). A novel variant of bubble sort with improved best-case performance. Journal of Discrete Algorithms, 2(7), 30–32. https://www.sciencedirect.com/science/article/pii/S1546221822006518

Research Publish. (s. f.). Performance comparison of sorting algorithms on the basis of complexity. Research Publish Journals. https://www.researchpublish.com/upload/book/Performance%20Comparison%20of%20Sorting%20Algorithms%20On%20The%20Basis%20Of%20Complexity-542.pdf

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Silas Faki, M., Mu'azu, M. B., & Abdu, M. (2016). An empirical analysis of internal sorting algorithms. Journal of Advances in Mathematics and Computer Science, 16(2), 1–12. https://journaljamcs.com/index.php/JAMCS/article/view/283

Skiena, S. S. (2020). The algorithm design manual (3rd ed.). Springer.